



Cartograph

Software Requirements Specification — what the system must do, stated precisely enough to be tested.

OWNER	Jagath Manjunath
DOCUMENT	SRS (deliverable of SDLC Phase 2)
VERSION	3 — 7 September 2026
GOVERNED BY	Project Charter v2 (Phase 1) — its fifteen definition-of-done criteria are the authority for priority in this document
CONTENTS	45 functional • 27 non-functional • 16 data • 16 interface requirements

This is a reference document. It is read once and consulted afterwards. Its length is a measure of coverage, not of what has to be held in mind. Underneath all of it, Cartograph is six things: read a schema into a graph • find the relevant piece of it • find the paths through it • write SQL • check the SQL before running it • show your work. Everything here is plumbing around those six, or bookkeeping about them.

01 How to read this document

Requirements are stated as testable assertions, identified permanently, and marked by priority.

Identifiers

FR functional · **NFR** non-functional · **DR** data · **IR** interface. Numbers are permanent. A requirement that is later dropped keeps its number and is marked withdrawn; numbers are never reused, so a reference in a test or a commit message always resolves to the same thing.

Priority

CORE — required by a criterion in the definition of done (Charter v2 §9, criteria 1 to 15). If a Core requirement is not met, the project is not finished by its own written definition. There is no judgement involved in this marking: it is a mapping, and §07 shows the mapping in full.

SUPPORTING — genuinely in scope and genuinely wanted, but serving no criterion. Nothing here is optional by default; this marking exists so that a decision made under pressure is one that was already thought through calmly.

WHY THIS DEFINITION, AND NOT THE EARLIER ONE

Version 1 defined Core as "remove it and the project is no longer Cartograph" — a judgement, and therefore arguable at exactly the moment arguing is most dangerous. Tying it to the definition of done makes it checkable instead. The change was only safe once the charter's criteria were extended to cover trace accuracy and trace readability; under the original eleven criteria this definition would have demoted the two most important requirements in the project.

The rule this document obeys

Requirements state *what*, never *how*. Technology choices are fixed by the charter's constraints and are not re-argued here. Where a genuine design decision surfaced during analysis, it is recorded in Section 08 and deferred to Phase 3 rather than pre-empted.

02 System overview

One request travels a fixed pipeline; every stage contributes to the trace.

#	STAGE	WHAT HAPPENS
1	Schema ingestion once, then cached	Read the warehouse catalog for tables, columns, primary keys and foreign keys. Build a graph. Embed the names.
2	Retrieval	Match the question against the embedded schema; select a bounded candidate set of tables and columns.
3	Path finding	Traverse foreign-key edges to connect the candidates. Discover every valid path; select one by a stated rule; keep the rest.
4	Generation	Assemble a prompt containing only the subgraph and the selected path. Call the model. Receive SQL.

#	STAGE	WHAT HAPPENS
5	Validation	Parse the SQL. Reject anything that is not a read-only SELECT, references anything that does not exist, or does not join along the path that was given.
6	Execution	Run on a read-only connection under a timeout. Return rows.
7	Trace	Return, alongside the rows, the complete record of stages 2 through 6 — including the paths not taken.

03 Functional requirements

Forty-four capabilities, each stated as an assertion that a test can confirm or refute.

ID	PRIORITY	THE SYSTEM SHALL...
A • SCHEMA INGESTION		
FR-01	CORE	Connect to a Postgres database using read-only credentials supplied as configuration.
FR-02	CORE	Extract from the database catalog every table, column, column data type, primary key and foreign key constraint.
FR-03	CORE	Construct a graph in which tables and columns are nodes and foreign-key relationships are edges.
FR-04	SUPPORTING	Persist the extracted schema and graph so that they are not rebuilt on every question.
FR-05	SUPPORTING	Re-run schema ingestion on demand and replace the stored schema and graph.
FR-43	CORE	Accept foreign-key relationships from a declared overlay file, merge them with whatever the catalog declares — catalog first, overlay filling the gaps — and mark on every edge which source it came from. Promoted to Core on 7 September: TPC-DS declares no constraints at all, so on the project's own demonstration warehouse this is the mechanism that produces a graph, not the fallback for one.
B • SEMANTIC INDEX		
FR-06	CORE	Generate vector embeddings for table and column names, together with any available comments or descriptions.
FR-07	CORE	Store those embeddings such that a question can be matched against them by similarity.
C • RETRIEVAL		
FR-08	CORE	On receiving a question, identify a candidate set of relevant tables and columns rather than passing the full schema to the model.
FR-09	CORE	Record which schema elements were retrieved, with their relevance scores.
FR-10	CORE	Bound the size of the retrieved subgraph.
FR-41	SUPPORTING	Where a question term maps to several candidate columns, record the candidates considered and the one chosen.
FR-42	SUPPORTING	Detect when no adequate schema elements are found and return an explicit "cannot be answered from this schema" response rather than attempting SQL generation.
D • JOIN-PATH FINDING		
FR-11	CORE	Compute join paths connecting the retrieved tables by traversing foreign-key edges.
FR-12	CORE	Detect when more than one valid join path connects the same pair of tables.

ID	PRIORITY	THE SYSTEM SHALL...
FR-13	CORE	Record all discovered paths and select one to use, by a stated and documented rule.
FR-14	CORE	Flag multi-path cases as ambiguous in the output.
FR-40	CORE	Express each discovered join path in plain English describing the relationship it represents.
E • SQL GENERATION		
FR-15	CORE	Assemble a prompt containing only the retrieved subgraph and the selected join path.
FR-16	CORE	Call an LLM API to generate SQL from that prompt and the user's question.
FR-17	CORE	Record the prompt sent and the SQL returned.
F • VALIDATION		
FR-18	CORE	Parse generated SQL into a syntax tree before execution.
FR-19	CORE	Reject any statement that is not a read-only SELECT.
FR-20	CORE	Reject SQL referencing tables or columns absent from the schema.
FR-21	CORE	Return a clear error without executing when validation fails.
FR-39	CORE	Verify that the join conditions in the generated SQL correspond to the selected join path, and flag or reject any mismatch.
G • EXECUTION		
FR-22	CORE	Execute validated SQL on a read-only connection.
FR-23	CORE	Enforce a query timeout.
FR-24	CORE	Return the result set as a table.
FR-25	CORE	Handle execution errors without crashing, and report them.
H • TRACE		
FR-26	CORE	Return with every answer a trace containing the retrieved tables and columns, the join path chosen, the alternative paths found, the generated SQL, and a plain-English account of how the query was assembled.
FR-27	CORE	Persist the trace alongside the query.
I • INTERFACE		
FR-28	CORE	Accept a plain-English question.
FR-29	CORE	Display the result table.
FR-30	CORE	Display the trace panel beside the results.

ID	PRIORITY	THE SYSTEM SHALL...
FR-31	CORE	Render the schema graph with the chosen join path visually highlighted and alternative paths distinguishable.
FR-32	CORE	Display validation errors clearly.
FR-45	CORE	Distinguish, in the interface and in the trace, a selection the system made arbitrarily from one it made on a stated basis. A tie broken alphabetically is arbitrary and is surfaced prominently; a shortest-path win or a declared preference is a reasoned choice and is reported quietly. Every discovered path remains recorded and drawn in both cases.
FR-44	SUPPORTING	Present a set of example questions in the interface.
J • ACCOUNTS AND HISTORY		
FR-33	CORE	Support user accounts.
FR-34	CORE	Record each user's query history with its trace, and allow it to be viewed again.
K • CACHING AND LIMITS		
FR-35	SUPPORTING	Serve a cached answer for a repeated identical question.
FR-36	SUPPORTING	Rate-limit requests per user.
L • EVALUATION		
FR-37	CORE	Run a fixed question set and record, per question, whether the retrieved tables and the chosen join path match the expected ones.
FR-38	CORE	Run that evaluation in two modes: graph retrieval, and full-schema prompting.

SCOPE OF THE EVALUATION

FR-37 measures whether retrieval selected the correct tables and the correct join path. It does **not** measure whether the final answer was numerically correct. This is a deliberate limit and must be stated wherever the result is reported, so the claim is not overread.

04 Non-functional requirements

Every figure below is a proposed target, not a measured one — to be revised once, on evidence, with the revision recorded.

ID	PRIORITY	REQUIREMENT
PERFORMANCE		
NFR-01	SUPPORTING	A typical question returns end to end in ≤ 10 s at median and ≤ 20 s at the 95th percentile. Dominated by the model call, not by application code.
NFR-02	SUPPORTING	Retrieval and path finding together complete in ≤ 1 s, excluding the model call. This is the portion under the project's own control and the portion held to account.
NFR-03	SUPPORTING	A cached repeat question returns in ≤ 500 ms.
NFR-04	SUPPORTING	Schema ingestion over the demo warehouse completes in ≤ 30 s.
CAPACITY		
NFR-05	SUPPORTING	Handles a schema of at least 50 tables and 500 columns without retrieval latency degrading. Set above the demo warehouse deliberately, so the system is not tuned to one schema.
NFR-06	SUPPORTING	A second question arriving while one is in flight is handled correctly — neither request fails, and neither returns the other's result.
SECURITY		
NFR-07	CORE	The warehouse connection uses a Postgres role granted SELECT only, with no DDL or DML privileges at the database level. Defence in depth: even were validation bypassed, the database itself refuses the write.
NFR-08	CORE	No execution path exists that reaches the database without passing FR-18 through FR-20.
NFR-09	CORE	Secrets — model API key, database credentials — are supplied by environment or configuration and never committed to the repository.
NFR-10	CORE	User passwords are stored hashed; sessions expire.
NFR-11	CORE	The query timeout is enforced at the database connection, not only in application code.
RELIABILITY		
NFR-12	CORE	A model API failure, timeout or malformed response produces a clear error — never a crash, never a partial or fabricated answer.
NFR-13	CORE	Any query that executed has its trace persisted. A trace is never lost.
COST CONTROL		
NFR-14	SUPPORTING	Every model call logs its token usage and estimated cost.
NFR-15	SUPPORTING	Retries are capped at two per question.

ID	PRIORITY	REQUIREMENT
NFR-16	SUPPORTING	Total model spend across the build stays within the charter's budget constraint (C-06).
NFR-25	SUPPORTING	The public deployment enforces a global spend ceiling; beyond it, cached answers only.
USABILITY		
NFR-17	CORE	The trace is comprehensible to someone who does not read SQL — plain English sits alongside the generated SQL, not instead of it.
NFR-18	CORE	The rendered graph stays legible: only the retrieved subgraph is drawn, never the full schema. This is a requirement, not styling — a hairball defeats the purpose of the panel.
NFR-19	SUPPORTING	The interface works in a current desktop browser at 1280px and wider. Mobile is not supported.
OPERABILITY		
NFR-20	CORE	The whole system starts with a single <code>docker compose up</code> on a clean machine.
NFR-21	CORE	The test suite runs on every push and completes in under five minutes.
NFR-22	SUPPORTING	Each pipeline stage — ingestion, retrieval, path finding, prompt assembly, generation, validation, execution — emits a structured log entry recording its duration in milliseconds, its inputs' identifiers, and its outcome. These durations are the evidence for NFR-01 through NFR-05.
NFR-26	CORE	The validation layer and the path-finding layer have explicit tests including negative cases — SQL that must be rejected, and SQL that joins differently from the given path while appearing to conform. Without this clause, a suite of one trivial test satisfies NFR-21 and criterion 9.
NFR-27	CORE	A single command verifies the system end to end from a clean start: the stack comes up, the schema ingests, one known question runs the full pipeline, and a result and trace are produced. It exits pass or fail. This is the green light every checkpoint is measured against.
PORTABILITY		
NFR-23	CORE	Runs identically on the development laptop and on the deployment host.
NFR-24	CORE	The deployed footprint fits within free-tier memory and storage limits.

05 Data requirements

The warehouse is not an implementation detail — the thesis depends on specific properties of it.

ID	PRIORITY	REQUIREMENT
THE WAREHOUSE UNDER QUERY		
DR-01	CORE	The demo warehouse is TPC-DS, loaded into Postgres from a documented, reproducible script. Verified 7 September: 24 tables — 7 fact, 17 dimension.
DR-02	CORE	Every conceptual relationship is available to the graph builder. Verified 7 September: the TPC-DS DDL declares no primary keys and no foreign keys of any kind. The relationships are therefore taken from the toolkit's published <code>tpcds_ri.sql</code> — roughly eighty statements — loaded into the overlay (FR-43) as the guaranteed source. The load script <i>also</i> attempts to apply them as real constraints, after adding primary keys to the seventeen dimension tables; where that succeeds the catalog declares them and FR-02 reads them directly.
DR-03	CORE	The warehouse provably contains at least one pair of tables connected by two or more distinct foreign-key paths — the sale-to-region ambiguity the thesis depends on. Verified explicitly, not assumed.
DR-04	CORE	Row volume is sufficient that aggregate queries return non-trivial, plausible results.
DR-05	CORE	Total warehouse size fits inside free-tier storage limits.
DR-06	CORE	The warehouse can be rebuilt from scratch by one documented command.
THE APPLICATION'S OWN STORE		
DR-07	CORE	User accounts: identifier, hashed password, creation time.
DR-08	CORE	Queries: question text, user identifier , timestamp, outcome status. User identity is present from the first migration, not retrofitted.
DR-09	CORE	Traces, one per query, holding: retrieved tables and columns with relevance scores and edge source; every join path discovered with its plain-English description; the path selected and the rule applied; column candidate sets where a term was ambiguous; the assembled prompt; the generated SQL; the validation outcome; the path-conformance outcome; execution duration; token usage with estimated cost, and the identifier of the user the query belongs to.
DR-10	SUPPORTING	Schema snapshots: extracted tables, columns, keys and the derived graph, persisted so that FR-04 holds.
DR-11	CORE	Embeddings for schema elements, stored alongside the schema snapshot they belong to.
DR-12	SUPPORTING	Cached answers keyed by question, with an expiry.
DR-13	CORE	The evaluation set: each question with its expected tables and expected join path, plus the recorded results of each run under each mode.
DR-14	SUPPORTING	All application schema changes are applied through migrations; no manual schema edits.

ID	PRIORITY	REQUIREMENT
DR-16	CORE	The evaluation question set and its expected answers are authored and committed before retrieval tuning begins, and an expected answer is never edited to match observed behaviour. A badly posed question is deleted and the deletion stated; it is not quietly corrected.
DR-15	SUPPORTING	No warehouse row data is copied into the application store beyond what is needed to display a result.

DR-09 IS THE LARGEST STRUCTURE IN THE PROJECT

It is what criterion 2 of the definition of done is checked against, and what the trace panel renders from. It is a first-class object, not a log blob. Its field-by-field shape is designed in Phase 3.

06 Interface requirements

The contract between the parts, and with the two services the system does not own.

ID	PRIORITY	REQUIREMENT
BACKEND API		
IR-01	SUPPORTING	The backend exposes an HTTP JSON API, versioned under a path prefix.
IR-02	CORE	A question endpoint accepts the question text and returns a query identifier together with the full result. The exchange is synchronous — one request, one response.
IR-03	CORE	An answer response carries status, the result set as columns and rows, and the trace object.
IR-04	CORE	The trace object carries every field enumerated in DR-09. This is the most important interface in the project.
IR-05	CORE	Errors return a machine-readable code alongside a human-readable message, distinguishing: validation failure, path-conformance failure, execution failure, model failure, and question-not-answerable.
IR-06	CORE	A schema endpoint returns the graph as nodes and edges for rendering.
IR-07	SUPPORTING	A re-ingest endpoint triggers FR-05.
IR-08	CORE	History endpoints list a user's past queries and fetch any one of them with its full trace.
IR-09	CORE	Authentication endpoints for register, login and logout.
IR-10	CORE	The evaluation set can be run in either mode and its results retrieved.
IR-11	SUPPORTING	Every request and response body is validated against a declared schema at the boundary. Nothing untyped crosses the API.
USER INTERFACE		
IR-12	SUPPORTING	While a question is in flight, the interface indicates which stage is running. At ten seconds a bare spinner reads as broken; a named stage reads as working — and it makes the pipeline legible in the demo video without narration.
IR-16	SUPPORTING	All frontend access to the API passes through a single client module; there are no scattered request call sites. Supporting by mapping, but early by sequence — it is what makes authentication a bolt-on rather than an edit of every call site, and it costs nothing if done first.
EXTERNAL SERVICES		
IR-13	SUPPORTING	The model provider is reached through a single adapter, so the provider can be changed without touching pipeline code.
IR-14	SUPPORTING	The embedding model is reached through a comparable adapter.

ID	PRIORITY	REQUIREMENT
IR-15	SUPPORTING	The warehouse connection is supplied at runtime as configuration. Nothing about a specific database is compiled in.

07 Traceability

Every criterion maps to requirements; every Core requirement serves a criterion. This table is the priority marking.

DOD	CRITERION (CHARTER V2 §9)	SATISFIED BY
1	Plain-English question returns a correct result table	FR-01-03, FR-06-11, FR-15-17, FR-22, FR-24, FR-25, FR-28, FR-29, FR-43 · NFR-12 · DR-01, DR-02, DR-04, DR-11 · IR-02, IR-03
2	Every answer carries a trace of retrieval, path chosen, alternatives	FR-09, FR-13, FR-17, FR-26, FR-27 · NFR-13 · DR-09 · IR-03, IR-04
3	Graph renders with the chosen path highlighted	FR-03, FR-31 · NFR-18 · IR-06
4	Non-read-only SQL rejected before execution, demonstrably	FR-18-21, FR-23, FR-32 · NFR-07, NFR-08, NFR-11 · IR-05
5	A demo question surfaces two valid paths and reports which was taken	FR-12, FR-14, FR-40 · DR-03
6	Evaluation run under both conditions, recorded in the README	FR-37, FR-38 · DR-13, DR-16 · IR-10
7	Whole system starts from a single command on a clean machine	NFR-20, NFR-27 · DR-06
8	Deployed and reachable at a public URL	NFR-23, NFR-24 · DR-05
9	Tests run on every push and pass	NFR-21, NFR-26
10	README documents architecture and decisions, alternatives included	FR-13 (stated rule) · Charter v2 §7
11	A demo video exists and is publishable	FR-31
12	Register, log in, see only your own history; sessions expire	FR-27, FR-33, FR-34 · NFR-10 · DR-07, DR-08 · IR-08, IR-09
13	Executed SQL provably joins along the path the trace reports	FR-18, FR-39 · NFR-26 · IR-05
14	A non-SQL reader can tell what each path means and which was taken	FR-30, FR-40, FR-45 · NFR-17
15	No credential anywhere in the repository or its history	NFR-09

THE REQUIREMENTS THAT MAP TO NOTHING

FR-04, FR-05, FR-35, FR-36, FR-41, FR-42, FR-44 · NFR-01 to NFR-06, NFR-14 to NFR-16, NFR-19, NFR-22, NFR-25 · DR-10, DR-12, DR-14, DR-15 · IR-01, IR-07, IR-11 to IR-16.

Thirty-one requirements serve no criterion. They are in scope because the founding documents named them or because analysis justified them, and none is intended to be dropped. But the absence of a mapping is itself the information: by the project's own definition of finished, losing any of these would not make it unfinished. **This is the slack, identified in advance rather than discovered under pressure.**

TWO ENTRIES ON THAT LIST DESERVE A SECOND LOOK

FR-43 is no longer on this list — the suspicion recorded here in version 2 was confirmed on 7 September and it is now Core. It is left mentioned because the pattern is worth noticing: a requirement that looks like a fallback is often the primary path once the world is checked. **IR-16** (single API client) is Supporting by mapping but must be done *early* by sequence — it is what keeps authentication a bolt-on. Neither is a candidate for dropping; both are mismarked by an honest rule rather than by an oversight, and saying so is better than bending the rule.

08 Decisions deferred to design

Surfaced during analysis, deliberately not pre-empted here.

REF	QUESTION	POSITION
FR-13	Which path is selected when several are valid?	Recommended: shortest path, ties broken deterministically, rule documented. A cleverer heuristic costs effort and weakens the thesis — the smarter the picker looks, the less a user feels they must check the trace.
DR-09	Exact field-by-field shape of the trace object	Fixed in Phase 3. This SRS fixes what must be in it, not how it is structured.
FR-08	How relevance is scored, and how the candidate set is bounded	Open. The genuine engineering problem of the project (Charter R-02); expected to need tuning against the evaluation set rather than settling on first attempt.
FR-33	Depth of the account system	Resolved 6 September. Full accounts — registration, login, hashed passwords, expiring sessions. Owner decision against the recommendation; see Charter v2, D-07. Consequence: criterion 12 exists, the auth requirements are Core, and authentication is built <i>after</i> the pipeline runs end to end.
—	Checkpoint ritual and build sequence	Named deliverables of Phase 3 , not afterthoughts within it. Checkpoints sit at deliverable boundaries, which do not exist until design decomposes the work. Proposed shape, owner-run: run NFR-27 from a clean start · run the test suite · tag the commit · answer in one line each — what works now that did not before, what was built that is not yet understood, what is the next single deliverable. The tag is the recovery mechanism; nothing can then cost more than the work since the last green state.

09 Threats and open suspicions

Written plainly: what can go wrong, what can be missed during the build, and what is outside anyone's control here.

The charter's risk register covers project-level risk. This section is narrower and more uncomfortable — it records the things most likely to be got *wrong quietly*. Each entry says what it is, why it may be missed, and what would surface it.

Threats to correctness

T-01 • CONFIRMED, THEN LARGELY RETIRED

A wrong edge is invisible — but the edges no longer have to be written

Confirmed 7 September: TPC-DS declares no primary keys and no foreign keys. The threat as originally written — that roughly thirty relationships would be transcribed by hand and a single wrong one would silently corrupt the graph — was real.

What retired it: the toolkit publishes `tpcds_ri.sql`, roughly eighty ready-made constraint statements written by the benchmark's own authors. The relationships are never authored, so they are never mis-authored.

What remains: the failure mode itself is unchanged in character — a wrong edge produces a path that should not exist, and the trace reports it with full confidence, with nothing erroring. So the verification test stays: **assert the expected edge count and the expected set of tied-path pairs over the loaded warehouse**, so the graph is checked rather than trusted. It is now a cheap confirmation rather than the only defence.

T-02 • STRUCTURAL

Path-conformance checking is weaker than it sounds

FR-39 compares the joins in the generated SQL against the selected path. Real SQL makes that comparison harder than it appears: table aliases, subqueries, derived tables, implicit joins in the WHERE clause, and self-joins can all produce SQL that passes a naive comparison while joining differently in substance.

Why it may be missed: the check will be written, it will pass on the demo questions, and it will look like a guarantee. It is a mitigation, not a guarantee.

What surfaces it: deliberately write SQL that should fail the check — an aliased join, a subquery join — and confirm it does. If it passes, the checker is decorative.

TPC-DS column names are abbreviations, and abbreviations embed badly

Semantic retrieval matches the question against schema names. TPC-DS names are compressed — `ss_ext_sales_price`, `ca_address_sk`, `d_moy`. The distance between the word "revenue" and the token `ss_ext_sales_price` in embedding space is not small. This is the cost of choosing TPC-DS over Pagila, whose names read like English, and it lands squarely on the project's hardest component.

Why it may be missed: retrieval will return *something* for every question. Poor retrieval looks identical to good retrieval until measured.

What surfaces it: the evaluation set — which is precisely why it must be authored early rather than at the end. Likely remedies if it bites: expand abbreviations into readable names before embedding, embed table context alongside the column, or attach column descriptions as database comments and embed those.

The evaluation is authored by the same person tuning the system

The question set defines what "correct retrieval" means, and the same hand will adjust retrieval until the numbers improve. Without discipline this measures nothing — the target moves with the arrow.

Why it may be missed: it never feels like cheating from the inside. Each individual adjustment is reasonable.

What surfaces it: write and freeze the question set and its expected answers *before* tuning begins, commit it, and never edit an expected answer to match observed behaviour. If a question turns out to be badly posed, delete it and say so — do not quietly correct it.

Threats to delivery**The public deployment may not be able to host the warehouse**

Definition of done criterion 8 requires a public URL. Constraint C-04 restricts hosting to free tiers. A TPC-DS warehouse with meaningful row volume may not fit a free-tier database, and the two requirements then contradict each other.

Why it may be missed: it does not appear until deployment, which is late.

What surfaces it: check free-tier storage limits against the generated warehouse size early — this is a five-minute check, not a build task. Remedies: generate at a smaller scale factor for the deployed instance, or deploy with a reduced warehouse and demonstrate the full one locally, stating the difference honestly.

T-06 • METHOD

Code that runs is not code that is right, and the gap is currently hard to see

The build method is to construct quickly with assistance and understand afterwards. That works, but it has a specific failure mode: a component that runs, produces plausible output, and is subtly wrong will not announce itself, and at present the reviewer of that code is also learning it.

Why it may be missed: plausible output is the most convincing kind of wrong.

What surfaces it: the evaluation set, the tests, and the trace itself — the trace is inspectable by design, so a wrong join path is visible in a way that most bugs are not. This is a case where the project's own thesis protects its builder.

T-09 • NEW, AND ALREADY CONTAINED

The constraints may refuse to apply to generated data

TPC-DS data generation is known to produce rows whose surrogate keys do not resolve to a dimension row, and there is a documented history of foreign key application failing on generated warehouses. Primary keys must also be added to the seventeen dimension tables first, since a foreign key needs a unique target.

Why this is contained rather than dangerous: the relationships are loaded into the overlay *first*, and the graph is built from there. Applying them to the database is attempted second and is optional to the mechanism. If it fails, nothing downstream changes except a line in the trace recording that the edges came from the overlay rather than the catalog — which the trace was already designed to say.

T-08 • INTRODUCED BY THE AUTHENTICATION DECISION

Cookies across two origins is the most reliable half-day sink in the plan

The frontend and the backend run on different origins. A session credential that must survive SameSite rules, credentialed requests and CORS preflight is a well-known source of failures that are opaque rather than instructive — the request simply arrives without the credential, and nothing explains why. This risk did not exist before authentication was committed.

Why it may be missed: it will not be missed. It will be found, loudly, at the worst moment — which is why it is here.

What contains it: build authentication only after the pipeline runs end to end, so this afternoon is spent against a working system rather than a half-built one. Route every request through IR-16's single client, so the fix lands in one place. And decide the credential mechanism deliberately in design rather than discovering it in the browser console.

Environment failures, provider behaviour, and the days themselves

WSL2, Docker and first-time database setup fail in ways unrelated to the project and consume time without producing progress. Model APIs change behaviour, rate-limit, or slow down without warning. Free-tier hosts impose limits that were not documented where they were looked for. And the nine days include learning that cannot be scheduled.

What this means in practice: none of these is a reason to change the plan, but the plan should not assume they are absent. The buffer against all of them is the same: build the risky and unfamiliar parts early, while there is still room to be wrong about them.

THE HONEST SUMMARY

T-01 was confirmed and retired on the day it was checked — verification before building is what made that cheap. **T-03 is now the single most likely to bite**, and it too is discoverable in the first day rather than the last. T-08 is certain rather than likely, and its cost is set entirely by *when* it is met. T-04 and T-06 are matters of discipline rather than difficulty. Everything here is known, named, and has something specific that would reveal it — which is a materially better position than the same project without this page.

10 Status and next step

PHASE	STATUS	NOTE
1 • Planning & feasibility	Closed	Charter signed off 5 September 2026.
2 • Requirements	Closed	This document, at version 2. All open inputs resolved.
3 • Design	Closed	Software Design Document v2 — 20 decisions, ordered build sequence, checkpoint ritual.
4 • Build	Next	Fourteen deliverables. Step 2 loads the warehouse and verifies the graph.

IMMEDIATE NEXT ACTION

Begin Phase 3. Its first task is the one everything else waits on: stand the warehouse up and verify that the graph it produces matches the TPC-DS specification — T-01 answered before anything is built on top of it.

11 Revision history

VERSION	DATE	CHANGE
1	6 Sep 2026	Issued. 44 functional, 25 non-functional, 15 data, 15 interface requirements. Priority marked by judgement.
3	7 Sep 2026	Verification pass against the TPC-DS specification. FR-43 promoted to Core — TPC-DS declares no constraints, so the overlay is the mechanism that produces a graph rather than a fallback for one. FR-45 added: the interface must distinguish an arbitrary selection from a reasoned one, so that the ambiguity

VERSION	DATE	CHANGE
		warning keeps its meaning on a graph with roughly eighty edges. DR-01 and DR-02 restated with verified facts — 24 tables, no declared constraints, relationships taken from the published tpcds_ri.sql. T-01 confirmed and largely retired; T-09 added for constraints failing to apply to generated data. Traceability regenerated.
2	6 Sep 2026	Priority redefined as a mapping to the charter's definition of done and every marking re-audited against the fifteen criteria — several requirements changed band as a result, including FR-04 and FR-43 downward and the authentication set upward. Added NFR-26 (tests of the validation and path-finding layers, negative cases included), NFR-27 (end-to-end smoke check), DR-16 (evaluation set frozen before tuning), IR-16 (single frontend API client). Amended DR-08 and DR-09 to carry user identity from the first migration. Resolved FR-33 in favour of full authentication. Added T-08 (cookies across origins). Traceability table regenerated for fifteen criteria; the unmapped-requirements note rewritten and two honest mismarkings called out rather than bent.